

New National Advertising

Replit Prompt: Full Admin Panel + Product Management

Copy the prompt below into Replit Agent. Build a real, secure, expandable admin dashboard. The first fully functional module is Products, allowing the administrator to add product information and images from the admin panel and have them appear on the public Products page.

MASTER REQUIREMENT

- Build a complete production-ready Admin Panel for the existing New National Advertising website.
- Do NOT rebuild the public website and do NOT break existing routes, service pages, More Info links, Products route, navbar, WhatsApp, Call buttons, quote flow, AI Assistant, Machines page, logo, address, or responsive behavior.
- Work incrementally on the existing codebase.

1. ADMIN ACCESS

- Create `/admin` and a secure `/admin/login` flow.
- Protect all admin routes and redirect unauthenticated users to `/admin/login`.
- Use the existing authentication setup if one already exists. Do not create a conflicting auth system.
- Never expose private API keys, database credentials, Firebase service-account keys, or other secrets in frontend code.
- Use proper server/backend authorization for mutations where the architecture supports it. Do not rely only on a hidden frontend button or client-side `isAdmin` flag.

2. LARGE PROFESSIONAL DASHBOARD

- Create a substantial, polished dashboard with a desktop sidebar and responsive mobile navigation.
- Use the existing New National Advertising visual language: white/light grey surfaces, navy typography, blue primary accent, restrained orange/yellow accents, clean cards, moderate radius, thin borders, subtle shadows and good whitespace.
- Include dashboard header, page title, administrator/account area, logout, loading states, empty states, error states and success feedback.
- Make the admin responsive from approximately 320px mobile through large desktop.

3. ADMIN NAVIGATION

- Dashboard
- Products
- Services
- Orders
- Website Content
- Machines
- Settings

4. PRODUCTS IS THE FIRST FULLY FUNCTIONAL MODULE

- The administrator must be able to add products without editing React/TypeScript code.
- Products created in the admin panel must be stored persistently and displayed on the public `/products` page.
- Do not populate the public page with invented products, prices, stock or descriptions.

5. ADD PRODUCT

- Create an Add Product form.
- Fields: Product Name, Short Description, Full Description, Product Image, Category, Price (optional), Product Status, Stock Status, and optional Display Order.
- Validate required fields.
- Show image preview before saving.
- Show loading/upload progress where supported.
- Prevent accidental duplicate submissions.

6. PRODUCT IMAGES

- Use the existing project storage solution, preferably the existing Firebase Storage setup if Firebase is already configured.
- Store a durable image URL/path in the product record, not a temporary local upload path.
- Allow replacing and removing product images safely.
- Use meaningful image alt text based on the product name.
- Optimize public product images without visibly damaging quality.

7. PRODUCT DESCRIPTIONS

- Allow the admin to enter a short description for product cards.
- Allow a longer description for a product detail view if the existing/public design supports one.
- Render saved product descriptions from the persistent data source.
- Do not hard-code product descriptions after this system is active.
- Use safe text rendering and do not introduce unsafe HTML injection.

8. PRODUCT MANAGEMENT LIST

- Create an admin Products table/grid.
- Show thumbnail, product name, category, price if present, status/stock status and last updated information.
- Add search, useful filters and sorting.
- Add Edit and Delete actions.
- Use a confirmation dialog before deletion.
- Provide a clean empty state.

9. EDIT PRODUCT

- Edit must open a populated form.
- Allow changing name, descriptions, image, category, price, status, stock status and display order.
- Save changes to the persistent data source and make them appear on /products.
- Do not lose an existing image when editing only text fields.

10. DELETE PRODUCT

- Require explicit confirmation.
- Show which product will be deleted.
- Handle errors gracefully.
- Clean up the associated image where safely supported, without deleting the wrong asset.

11. PUBLIC PRODUCTS PAGE

- Keep the existing /products route.
- Products added through the admin panel must appear there.
- Edited products must update there.
- Deleted products must disappear there.
- Keep the current public design consistent with the website.
- Do not add fake product data.

12. PRODUCT CATEGORIES + STATUS

- Use a reusable category structure and allow products to be assigned to categories.
- Support simple statuses such as In Stock, Low Stock and Out of Stock.
- Do not invent stock quantities unless an actual quantity field is implemented.
- Keep the category system expandable.

13. DASHBOARD OVERVIEW

- Show real summary cards such as Total Products, Published Products, Inactive/Draft Products and Categories.
- Do not show fake statistics.
- Keep the dashboard ready for future Orders, Content and other modules.

14. SERVICES FOUNDATION

- Include Services in the admin navigation as a structured foundation.
- Do not change current public service names, order, routes or content unless explicitly instructed.
- Current public order: Signage Board, Banner Printing, Eco Solvent Flex, Offset Printing, Screen Printing, Graphics Design, Digital Printing.
- Keep existing route slugs stable, including /services/sign-boards and /services/solvent-flex.

15. WEBSITE CONTENT FOUNDATION

- Create a Website Content section that can later manage homepage and contact content.
- For this stage, do not migrate every hard-coded website element unless needed.
- Keep it modular for future expansion.

16. MACHINES FOUNDATION

- Include Machines in the admin navigation as a foundation for future machine management.
- Do not invent machine data or change the existing Machines page unless already implemented.
- Prepare the architecture for future machine images, titles and descriptions.

17. SECURITY + DATA INTEGRITY

- Protect every create/edit/delete action.
- Validate incoming data.
- Use authenticated admin checks on the backend/server where applicable.
- Never expose private credentials in browser code.
- Prevent unauthorized users from creating, editing or deleting products.

18. UX + ERROR HANDLING

- Show clear success/error feedback.
- Disable submit buttons during operations.
- Use confirmation dialogs for destructive actions.
- Maintain keyboard accessibility, visible focus states and reduced-motion support.

19. EXISTING WEBSITE MUST NOT BREAK

- Do not rewrite App.tsx unnecessarily.
- Do not replace the router.
- Do not rename existing public URLs.
- Do not remove the Products page.
- Do not break More Info links.
- Do not break WhatsApp, Call, Get a Quote, AI Assistant, Machines, logo, address, navbar or mobile navigation.
- Do not make unrelated UI changes.

20. FINAL TESTING

- Test /admin/login and unauthorized /admin access.
- Test successful admin login and logout.
- Add a real test product with image and description, then verify it appears on /products.
- Edit the product and verify the public page updates.
- Delete the test product and verify it disappears.
- Test image replacement and validation/error states.
- Test desktop and mobile admin layouts.
- Test all existing public routes and service More Info links.
- Run TypeScript/typecheck and the production build.
- Fix implementation errors before finishing.

EXPECTED PRODUCT WORKFLOW

Admin login → Products → Add Product → enter product name and description → upload product image → save → persistent storage/database → public /products reads the saved data → customers see the product image and description. Editing or deleting from the admin panel must update the public Products page.

IMPLEMENTATION PRINCIPLE

Build a real management system, not a visual mockup. The Products module must be functional and connected to persistent data/storage. Keep the architecture secure, modular and expandable because additional admin features will be added later.